Name:Thanmayee A J

Date:17-03-2026

Objective:

In today's session, I learnt how to build a RESTful API using Node.js and Express.js. I understood how to connect a MongoDB database using Mongoose and create a product schema. I implemented CRUD operations such as adding, viewing, updating, and deleting products. I also tested the APIs using Postman and verified the data in MongoDB Compass.

Task:

db.js

```
import mongoose from "mongoose";
const connectDB = async () => {
   try {
      await mongoose.connect(process.env.MONGO_URI);
      console.log("mongodb connected");
   } catch (error) {
      console.error("database connection failed");
      process.exit(1);
   }
};
export default connectDB;
```

products.js

```
import mongoose from "mongoose";
const productSchema = new mongoose.Schema({
name: {
type: String,
required: true
   },
   price: {
      type: Number,
      required: true
```

```
    },
    category: {
        type: String
    },
    stock: {
        type: Number,
    }
}, { timestamps: true });
const Product = mongoose.model("products", productSchema);
export default Product;
```

productroutes1.js

```
import express from "express";
import Product from "../models/products.js";
const router = express.Router();
router.post("/products", async (req, res) => {
    try {
        const product = new Product(req.body);
        await product.save();
        res.status(201).json(product);
    } catch (error) {
        res.status(500).json({ message: error.message });
    }
});
router.get("/products", async (req, res) => {
    try {
        const products = await Product.find();
        res.json(products);
    } catch (error) {
        res.status(500).json({ message: error.message });
    }
```

```
});
router.get("/products/:id", async (req, res) => {
    try {
        const product = await Product.findById(req.params.id);
        if (!product) {
            return res.status(404).json({ message: "user not found" });
        }
        res.json(product);
    } catch (error) {
        res.status(500).json({ message: error.message });
    }
});
router.put("/products/:id", async (req, res) => {
    try {
        const updatedProduct = await Product.findByIdAndUpdate(
            req.params.id,
            req.body,
            { new: true }
        );
        if (!updatedProduct) {
            return res.status(404).json({ message: "product not found" });
        }
        res.json(updatedProduct);
    } catch (error) {
        res.status(500).json({ message: error.message });
    }
});
router.delete("/products/:id", async (req, res) => {
    try {
        const deletedProduct = await Product.findByIdAndDelete(req.params.id);
```

```
        if (!deletedProduct) {
            return res.status(404).json({ message: "product not found" });
        }
        res.json({ message: "product deleted successfully" });
    } catch (error) {
        res.status(500).json({ message: error.message })
    }
});
export default router;
```

.env

```
PORT = 5000
MONGO_URI = mongodb://localhost:27017/products
```

server1.js

```
import express from "express";
import dotenv from "dotenv";
import connectDB from "./config/db.js";
import productroutes1 from "./routes/productroutes1.js";
dotenv.config();
connectDB();
const app = express();
app.use(express.json());
app.use("/api", productroutes1);
const PORT = process.env.PORT;
app.listen(PORT, () => {
console.log(`server is running on port ${PORT}`);
})
```

Output:

```
_id: ObjectId('69b921f16b0a8c9abe4da442')
name : "Laptop"
price : 50000
category : "electronics"
stock : 10
createdAt : 2026-03-17T09:42:09.459+00:00
updatedAt : 2026-03-17T09:42:09.459+00:00
__v : 0
```

POST http://localhost:5000/api/products Send

Params Authorization Headers (9) Body Scripts Settings Code Cooki

none form-data x-www-form-urlencoded raw binary GraphQL JSON Beauti

```
1 {
2     "name":"Laptop",
3     "price":50000,
4     "category":"electronics",
5     "stock":10
6 }
```

Body Cookies Headers (7) Test Results Status: 201 Created Time: 130 ms Size: 427 B

Pretty Raw Preview JSON


```
1 {"name":"Laptop","price":50000,"category":"electronics","stock":10,
    "_id":"69b921f16b0a8c9abe4da442","createdAt":"2026-03-17T09:42:09.459Z",
    "updatedAt":"2026-03-17T09:42:09.459Z","__v":0}
```


http://localhost:5000/api/products Save No Environment

GET http://localhost:5000/api/products Send

Params Authorization Headers (7) Body Scripts Settings Code Cookies

none form-data x-www-form-urlencoded raw binary GraphQL

This request does not have a body

Body Cookies Headers (7) Test Results Status: 200 OK Time: 11 ms Size: 424 B

Pretty Raw Preview

[{"_id":"69b921f16b0a8c9abe4da442","name":"Laptop","price":50000,"category":"electronics","sto
03-17T09:42:09.459Z","updatedAt":"2026-03-17T09:42:09.459Z","__v":0}]

http://localhost:5000/api/products/69b921f16b0a8c9abe4da442 Save No Environment

GET http://localhost:5000/api/products/69b921f16b0a8c9abe4da442 Send

Params Authorization Headers (7) Body Scripts Settings Code Cook

none form-data x-www-form-urlencoded raw binary GraphQL JSON Beauti

Body Cookies Headers (7) Test Results Status: 200 OK Time: 9 ms Size: 422 B

Pretty Raw Preview JSON


```
{"_id":"69b921f16b0a8c9abe4da442","name":"Laptop","price":50000,
"category":"electronics","stock":10,"createdAt":"2026-03-17T09:42:09.
459Z","updatedAt":"2026-03-17T09:42:09.459Z","__v":0}
```


PUT http://localhost:5000/api/products/69b921f16b0a8c9abe4da442 Send

Params Authorization Headers (9) Body Scripts Settings Code Cook

none form-data x-www-form-urlencoded raw binary GraphQL JSON Beauti


```
{
    "name":"Phone"
}
```


Body Cookies Headers (7) Test Results Status: 200 OK Time: 19 ms Size: 421 B

Pretty Raw Preview JSON


```
{"_id":"69b921f16b0a8c9abe4da442","name":"Phone","price":50000,
"category":"electronics","stock":10,"createdAt":"2026-03-17T09:42:09.
459Z","updatedAt":"2026-03-17T09:56:59.698Z","__v":0}
```


DELETE http://localhost:5000/api/products/69b921f16b0a8c9abe4da442 Send

Params Authorization Headers (7) Body Scripts Settings Code Cookies

Query Params

	Key	Value
	Key	Value

Body Cookies Headers (7) Test Results Status: 200 OK Time: 20 ms Size: 277 B

Pretty Raw Preview JSON


```
{"message":"product deleted successfully"}
```

ADD DATA | UPDATE | DELETE | EXPORT DATA | EXPORT CODE

25 0 – 0 of 0

This collection has no data

It only takes a few seconds to import data from a JSON or CSV file.

Import data